

project2_py\plotters.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 try:
4     from .project2 import optimize_with_history
5 except ImportError:
6     from project2 import optimize_with_history
7
8 def plot_contour(a, b, z, path=None, ax=None, draw_contour = True):
9     # Get range of function values
10    Z_min = np.min(z)
11    Z_max = np.max(z)
12
13    # Define the number of contour lines & power factor
14    num_levels = 100
15    power_factor = 3
16
17    positive_levels = Z_max * (np.linspace(0, 1, num_levels) ** power_factor)
18
19    # If there are values below zero, plot them
20    negative_levels = []
21    if Z_min < 0:
22        # Sample values below 0 for half of num_levels contours
23        negative_levels = abs(Z_min) * (np.linspace(0, 1, num_levels // 2) ** power_factor)
24        negative_levels = -np.sort(negative_levels)[::-1] # Sort in ascending order and flip to
negative
25
26    all_levels = np.unique(np.concatenate((negative_levels, positive_levels)))
27
28    if ax is None:
29        ax = plt.gca()
30        show_plot = True
31    else:
32        show_plot = False
33
34    if draw_contour: ax.contour(a, b, z, levels=all_levels, colors="#282853", linestyle='solid',
linewidths=0.5)
35
36    # If there's an optimization path, plot it
37    if path is not None:
38        path = np.array(path)
39        if draw_contour == True:
40            ax.plot(path[:, 0], path[:, 1], 'r', linewidth = 0.75, label="Optimization Path")
41            ax.plot(path[0, 0], path[0, 1], 'go', label = "Start")
42            ax.plot(path[-1, 0], path[-1, 1], 'm*', markersize=10, label="Finish")
43        else:
44            ax.plot(path[:, 0], path[:, 1], 'r', linewidth = 0.75)
45            ax.plot(path[0, 0], path[0, 1], 'go')
46            ax.plot(path[-1, 0], path[-1, 1], 'm*', markersize=10)
47
48    from matplotlib.lines import Line2D
49    custom_legend = [Line2D([0], [0], color="#282853", linewidth=0.5, label="Contour Lines")]
50    if path is not None:
```

```

51         custom_legend.extend([
52             Line2D([0], [0], color="r", linewidth=0.75, label="Optimization Path"),
53             Line2D([0], [0], color="g", marker="o", linestyle="None", label="Start"),
54             Line2D([0], [0], color="m", marker="*", markersize=10, linestyle="None", label="Finish")
55         ])
56     ax.legend(handles=custom_legend, fontsize='x-small')
57     ax.set_title("Contour Plot of Simple1 using QPM + ADAM")
58     ax.set_xlabel("A")
59     ax.set_ylabel("B")
60
61     if show_plot:
62         plt.show()
63
64
65 def plot_convergence(problem):
66     plt.figure()
67     for _ in range(3):
68         problem._reset()
69         path = optimize_with_history(
70             problem.f,
71             problem.g,
72             problem.c,
73             problem.x0(),
74             problem.n,
75             problem.count,
76             problem.prob
77         )
78         iters = range(len(path))
79         f_vals = np.zeros(len(path))
80         for idx, value in enumerate(path):
81             f_vals[idx] = problem.f(value)
82         plt.plot(iters, f_vals)
83     titles = [f'Run {i+1}' for i in range(3)] # More generic labels
84     plt.xlabel("Iterations")
85     plt.ylabel("Objective Value")
86     plt.title(f"Objective Value vs # Iterations for {problem.prob} using QPM")
87     plt.legend(titles)
88     plt.grid(True)
89     plt.show()
90
91
92 def plot_constraint(problem):
93     plt.figure()
94     for _ in range(3):
95         problem._reset()
96         path = optimize_with_history(
97             problem.f,
98             problem.g,
99             problem.c,
100             problem.x0(),
101             problem.n,
102             problem.count,
103             problem.prob
104         )

```

```

105     iters = range(len(path))
106     c_vals = np.zeros(len(path))
107     for idx, value in enumerate(path):
108         c_vals[idx] = np.max(problem.c(value))
109     plt.plot(iters, c_vals)
110     titles = [f'Run {i+1}' for i in range(3)] # More generic labels
111     plt.xlabel("Iterations")
112     plt.ylabel("Maximum Constraint Value")
113     plt.title(f"Max Constraint vs # Iterations for {problem.prob} using QPM")
114     plt.legend(titles)
115     plt.grid(True)
116     plt.show()
117
118
119
120 def plot_problem(problem, plot_size=3):
121     """
122     Visualizes 2D functions with optimization paths overlaid on contours.
123     """
124     def get_wrappedf(x1_val, x2_val):
125         x = np.zeros(problem.xdim)
126         if problem.xdim > 0: x[0] = x1_val
127         if problem.xdim > 1: x[1] = x2_val
128         return problem._wrapped_f(x)
129
130     def get_max_constraint(x1_val, x2_val):
131         x = np.zeros(problem.xdim)
132         if problem.xdim > 0: x[0] = x1_val
133         if problem.xdim > 1: x[1] = x2_val
134         return np.max(problem.c(x))
135
136     x1 = np.linspace(-plot_size, plot_size, 100)
137     x2 = np.linspace(-plot_size, plot_size, 100)
138     A, B = np.meshgrid(x1, x2)
139
140     vectorized_f = np.vectorize(get_wrappedf)
141     Z = vectorized_f(A, B)
142
143     vectorized_c = np.vectorize(get_max_constraint)
144     C_max = vectorized_c(A, B)
145
146     _, ax = plt.subplots()
147     plot_contour(A, B, Z, ax=ax, draw_contour=True)
148     ax.contourf(A, B, C_max, levels=[-np.inf, 0], colors=["#E1FF00"], alpha=0.5)
149     ax.contour(A, B, C_max, levels=[0], colors=['black'], linewidths = 1.5)
150
151     path = None
152     for _ in range(3):
153         problem._reset()
154         path = optimize_with_history(
155             problem.f,
156             problem.g,
157             problem.c,
158             problem.x0(),

```

```
159         problem.n,  
160         problem.count,  
161         problem.prob  
162     )  
163     print(f"Number of Steps Taken: {len(path)}")  
164     print(f"Total Count: {problem.count()}")  
165     plot_contour(A, B, Z, path=path, ax=ax, draw_contour=False)  
166  
167     print(f"Final constraints: {problem.c(path[-1])}")  
168  
169     plt.tight_layout()  
170     plt.show()  
171
```